



PA 2 Report

2024010779 刘家豪

实现方法

Kernel 架构

实现包含 4 个 kernel，采用分块计算策略：

- `phase1`：处理当前 `pivot` 块的自更新
- `phase2_row`：更新 `pivot` 所在行的所有块
- `phase2_col`：更新 `pivot` 所在列的所有块
- `phase3`：并行更新剩余所有非 `pivot` 块

线程与线程块组织

参数配置： $B = 16$ ，每个线程块维度为 16×16 线程； $R = 4$ ，每个线程负责处理 4×4 个数据元素； $BS = B \times R = 64$ ，每个数据块大小为 64×64

线程块配置：`dim3 thr(B, B)`，256 个线程/块

块网格分配策略：`phase1` 为 `<<<1, thr>>>`，使用单个线程块处理 `pivot` 块；`phase2_row` 和 `phase2_col` 为 `<<<num_blocks, thr>>>` 一维网格，每个块分别处理行和列中的一个块；`phase3` 为 `<<<dim3(num_blocks, num_blocks), thr>>>` 二维网格覆盖所有剩余块，跳过 `pivot` 行列

存储层次利用

- 每个线程维护 `int reg[R][R]` 寄存器数组，通过寄存器复用减少对共享内存和全局内存的访问次数。
- 共享内存分配如下：
 - `phase1`：单个 `shared[BS][BS]` 存储当前块的完整副本
 - `phase2_row`：双缓冲策略，`shared_pivot[BS][BS]` 存储 `pivot` 块，`shared_cur[BS][BS]` 存储当前处理的行块
 - `phase2_col`：类似双缓冲，存储 `pivot` 块和当前列块

- `phase3` : `shared_row[BS][BS]` + `shared_col[BS][BS]` 分别存储 `pivot` 的行和列数据
- 全局内存分配方式：图数据以一维数组 `graph[n*n]` 形式存储，采用合并访问模式，确保相邻线程访问连续内存地址，仅在初始加载和最终写回阶段访问全局内存，通过边界检查 `g_i + r_i < n && g_j + r_j < n` 避免越界访问。

计算流程与优化

共享内存读取预取

在 k 循环中，每个线程需要读取 `shared[posy][k]` (`pivot` 列的第 k 个元素，对固定 i) 和 `shared[k][posx]` (`pivot` 行的第 k 个元素，对固定 j)。这两个值在内部 r_i/r_j 循环中是不变的，因此预先提取到寄存器数组 `col_k[R]` 和 `row_k[R]` 中，将共享内存读取量从每 k 的 $2 \times R^2 = 32$ 次降低到 $2 \times R = 8$ 次，减少了 4 倍的共享内存带宽压力。

Phase 1 的共享内存写回

Phase 1 在 `pivot` 块内部执行 Floyd-Warshall 算法，其 k 循环需要迭代 $BS=64$ 次。由于 Floyd-Warshall 的递推性质：第 $k+1$ 次迭代中用到的 `d[i][k+1]` 和 `d[k+1][j]` 可能已被第 k 次迭代更新。因此 Phase 1 的 k 循环必须在每次迭代后将寄存器结果写回共享内存并执行 `__syncthreads()`，以确保所有线程看到最新的中间结果。

Phase 2/3 的 min-plus 矩阵乘

Phase 2 和 Phase 3 本质上是 min-plus 矩阵乘法 $C[i][j] = \min_k (A[i][k] + B[k][j])$ 。其中操作数矩阵 A 和 B 在 k 循环期间是静态的，不会被中间结果更新。因此 Phase 2/3 的 k 循环不需要共享内存写回和同步，仅需预取优化即可。

关于 `shared[BS][BS + 1]` 填充

尝试使用 `shared[BS][BS + 1]` 来消除列访问的 bank conflict，但测试发现性能显著下降。推测原因是：填充使 Phase 2/3 的共享内存从 32KB 增加到 32.5KB 每块，导致 GTX 1080 上的 occupancy 从每 SM 3 个块降到 2 个块，减少了 33% 的活跃 warp，抵消了 bank conflict 的消除收益。

实验结果

n	优化前运行时间 (ms)	优化后运行时间 (ms)	加速比
1000	15.38	1.70	9.05
2500	377.79	15.36	24.60
5000	2986.16	84.48	35.35
7500	10050.54	315.79	31.83
10000	22833.93	614.25	37.17